# Story space: salient concepts as volumes in a story-conditioned latent space

## 1 Problem

**Definition 1** (Story, paragraphs)**.** *A story $s \in \mathcal{T}$ is a string of text from the space of texts $\mathcal{T}$. It is split into paragraphs $p_1, \ldots, p_N \in \mathcal{T}$.*

| | |
|---|---|
| $s$ | the story, a string of text |
| $\mathcal{T}$ | the space of all strings of text |
| $p_i$ | the story's $i$-th paragraph |
| $N$ | the number of paragraphs in the story |

**Definition 2** (Salient concept)**.** *A salient concept $c \in \mathcal{T}$ is a string of text naming an idea the story contains. It need not be copied out of the story: the idea may never be mentioned directly. A story comes with a set of salient concepts $C_s = \{c_1, \ldots, c_K\}$.*

| | |
|---|---|
| $c$ | a salient concept, a string of text naming an idea |
| $C_s$ | the set of the story's salient concepts |
| $K$ | the number of salient concepts |

**Definition 3** (Story state)**.** *A story state $\sigma$ is a structured description of a moment of the story (characters, location, event, action). A fixed template $\tau$ renders it as text, $\tau(\sigma) \in \mathcal{T}$.*

| | |
|---|---|
| $\sigma$ | a story state: characters, location, event, action |
| $\tau$ | the fixed template that writes a state out as text |
| $\tau(\sigma)$ | the text of state $\sigma$ |

**Definition 4** (Story space)**.** *The story space $\mathcal{Z}_s \subseteq \mathbb{R}^d$ is the latent space of story $s$. Every salient concept $c$ is a volume $B_c \subset \mathcal{Z}_s$ and every story state $\sigma$ is a point $z(\sigma) \in \mathcal{Z}_s$. A state is related to a concept when its point lies inside the concept's volume. Two concepts are related when their volumes overlap.*

| | |
|---|---|
| $\mathcal{Z}_s$ | the story space of story $s$ |
| $d$ | the number of dimensions of the story space |
| $\mathbb{R}^d$ | the space of vectors of $d$ real numbers |
| $B_c$ | the volume of concept $c$ in the story space |
| $z(\sigma)$ | the point of state $\sigma$ in the story space |

Figure 1 shows two overlapping concept volumes and a story moving through them.

The story space is a conditioning of the general latent space of all texts: of all the places a text can occupy in the general space, the story picks out the region that it, its concepts and its states occupy.

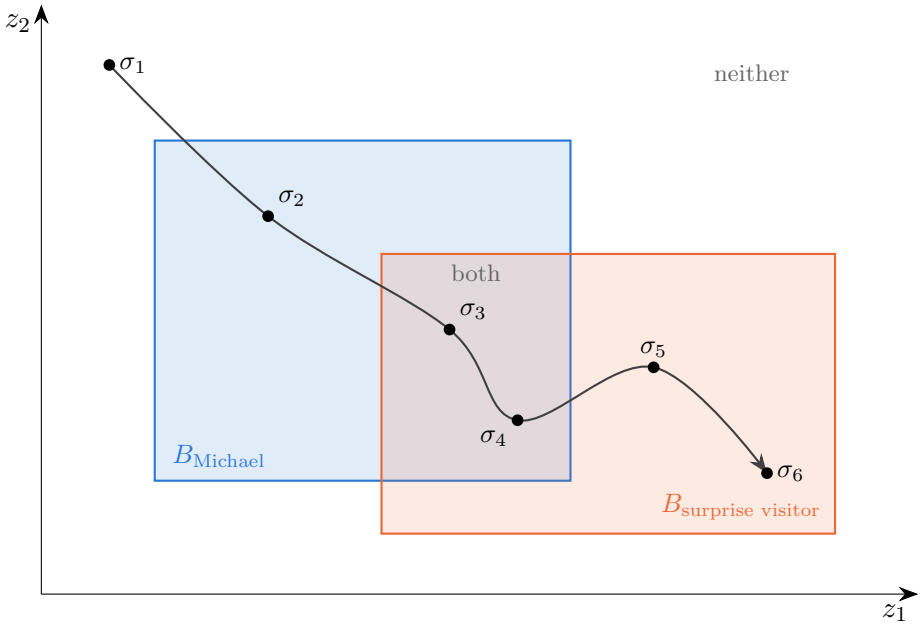



Figure 1: Illustration, not computed from data. Two concept volumes in a story space and the trajectory of SCENE_1's states in story order. $z_1$ and $z_2$ are two of the story space's $d$ dimensions; $\sigma_k$ marks the point of the story's $k$-th state. $\sigma_1$, a warm sunny day, lies in neither volume. $\sigma_2$, Michael and Sarah setting out for a walk, lies only in $B_{\text{Michael}}$. $\sigma_3$, an unexpected knock at their door, and $\sigma_4$, Michael going down to check, lie in the overlap. $\sigma_5$, Sarah calling "Who is it?", and $\sigma_6$, no answer coming, lie only in $B_{\text{surprise visitor}}$.

**What the model does.** The model takes a story as input and answers two questions about it:

```
class TextSpan:
  firstWordIndex: int
  lastWordIndex: int

class SalientConcept:
  name: str
  abstract: str
  references: list[TextSpan]

def salient_concepts(story: str) -> list[SalientConcept]

class Plot:
  name: str
  abstract: str
  causal_events_in_order: list[TextSpan]

def plots(story: str, concepts: list[SalientConcept]) -> list[Plot]
```

- `salient_concepts` takes a story and returns its salient concepts. Each concept has a name, an abstract that describes the idea, and the spans of the story that refer to it.

- `plots` takes a story and its salient concepts and returns the story's plots. Each plot has a name, an abstract, and the spans of the story that are its causal events, in story order.
- A `TextSpan` is a range of the story's words, the first and the last included, numbered as in Section 3.3.

These are two models. `plots` takes the concepts that `salient_concepts` returns, so it is a separate model, trained on its own data. The rest of this document is about `salient_concepts` only.

An annotated `.author` file holds exactly what `salient_concepts` returns for its story. Each salient-concept cell is one `SalientConcept`: the cell's title is the name, its text is the abstract, and its `words` ranges are the references.

**Requirements.**

**Requirement 1** (Non-degenerate volume)**.** *The volume of a concept has non-zero width along every dimension of $\mathcal{Z}_s$.*

**Requirement 2** (Containment)**.** *A state related to concept $c$ maps to a point inside $B_c$. A state unrelated to $c$ maps outside it.*

**Requirement 3** (Multiple membership)**.** *A state may be related to several concepts at once, so volumes may overlap and a point may lie in several of them.*

**Requirement 4** (Identity)**.** *Equal states map to the same point: $\sigma = \sigma' \Rightarrow z(\sigma) = z(\sigma')$.*

**Requirement 5** (Separation)**.** *Different states map to different points: $\sigma \neq \sigma' \Rightarrow z(\sigma) \neq z(\sigma')$.*

$\sigma'$ a second story state, compared with $\sigma$

## 2 How Qwen encodes a text

Qwen is used as it is: frozen, nothing in it trained. The figures were measured with Qwen3-8B, which has $L = 36$ layers and a latent space of $D = 4096$ dimensions.

**A point in latent space.** Qwen splits a text $t$ into tokens $k = 0, \ldots, n-1$. After each layer $\ell$ it holds, for every token, a point $h_{\ell,k}(t)$ in its latent space $\mathbb{R}^D$; $\ell = 0$ is the token's lookup before any layer.

**The latent space.** $\mathbb{R}^D$ is a real vector space with the dot product $u \cdot v = \sum_{i=1}^{D} u_i v_i$, which gives lengths, angles and distances. All layers share it. The points are not normalised, and their lengths are bounded: the longest here is about 12,000. The points crowd into a narrow cone whose apex is the space's zero, because training gives every point one large common component.

**One point per token.** Qwen gives one point per token, not one per text. Each point summarises the text up to and including its token. The text "Bob" gives one point after each layer; a passage of $n$ tokens gives $n$.

**A token's path through the layers.** Following one token from layer 0 to layer $L$ gives a trajectory of $L+1$ points. Each step is one layer: the layer analyses the point, attending to the points of the tokens before it and passing the result through a feed-forward network, and adds what it finds to the point:

$$h_{\ell,k}(t) = h_{\ell-1,k}(t) + \Delta_{\ell,k}(t). \tag{1}$$

**Where the information is.** In each point itself: in where it lies along the space's feature directions, not in how the points relate to each other.

**Token** 0**.** From layer 7 on, the first token's point becomes an attention sink: about 185 times longer than the others and the same for every text, whatever its first word. It is left out of everything below.

| | |
|---|---|
| $t$ | any text given to Qwen |
| $k$ | a token's position in the text, starting at 0 |
| $n$ | the number of tokens in the text |
| $\ell$ | a layer of Qwen; 0 means the lookup before the first layer |
| $L$ | the number of Qwen's layers |
| $D$ | the number of dimensions of Qwen's latent space |
| $\mathbb{R}^D$ | Qwen's latent space |
| $h_{\ell,k}(t)$ | the point in $\mathbb{R}^D$ that Qwen holds for token $k$ of text $t$ after layer $\ell$ |
| $u, v$ | any two points of $\mathbb{R}^D$ |
| $u_i$ | the $i$-th of the $D$ coordinates of $u$ |
| $u \cdot v$ | the dot product of $u$ and $v$ |
| $\Delta_{\ell,k}(t)$ | what layer $\ell$ adds to token $k$'s point |

## 2.1 Extending a text

Three texts share a beginning:

| | |
|---|---|
| A | "Bob buys Alice coffee" |
| A + B | "Bob buys Alice coffee, because he loves her" |
| A + C | "Bob buys Alice coffee and bagels with salmon" |

B adds Bob's motivation; C adds a detail that changes nothing important. Figure 2 shows their points after layer 16.

**Salient information.** Write $I(t)$ for the salient information a text $t$ carries, and $V(t)$ for the volume that surrounds text $t$. We take as given:

$$\begin{aligned} |V(A)| \approx |V(B)|, \qquad |V(A)| > |V(C)|, \\ |V(A+B)| \gg |V(A+C)|, \qquad |V(A)| \approx |V(A+C)|. \end{aligned} \tag{2}$$

What remains to be found is how Qwen encodes and represents this information.

| | |
|---|---|
| $I(t)$ | the salient information text $t$ carries |
| $V(t)$ | the volume that surrounds text $t$ |
| $\lvert V(t)\rvert$ | the size of that volume |

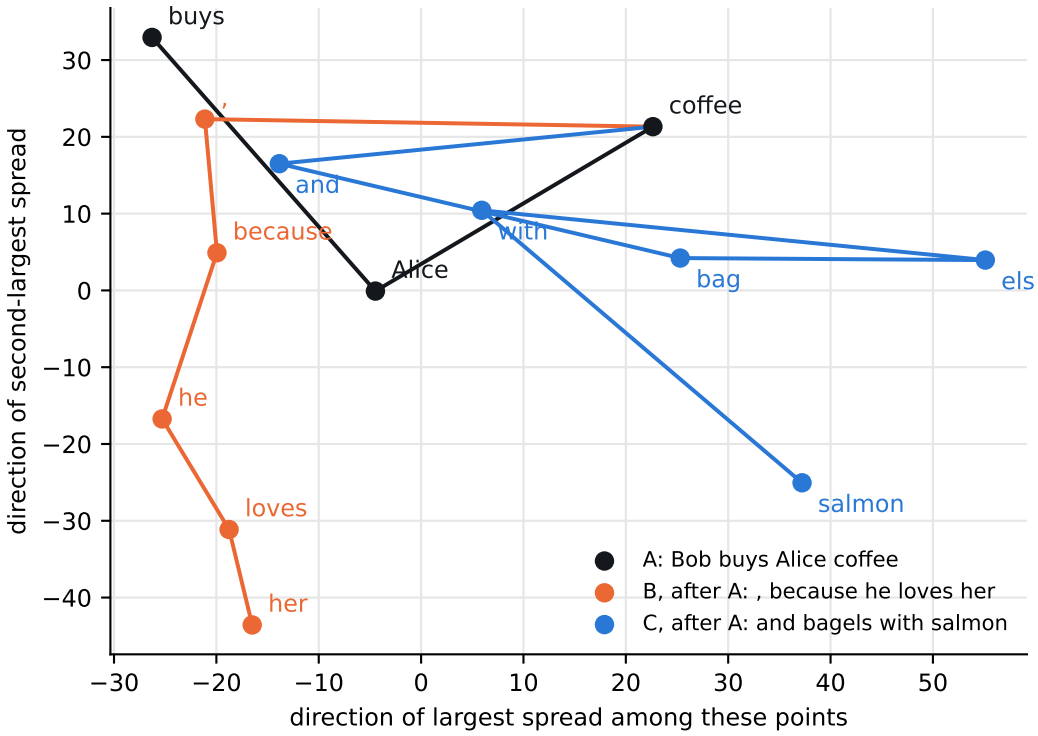



Figure 2: The points of A, A + B and A + C after layer 16, token 0 ("Bob") left out. Black: A's points, shared by all three texts. Orange: the points B adds after A. Blue: the points C adds after A. Lines join the points in reading order. The 13 points live in $\mathbb{R}^{4096}$; they are drawn along the two directions in which they differ most, which hold 34% of their spread.

**Shannon information.** The Shannon information of a text under Qwen is the number of bits Qwen needs to predict it, token by token:

$$H(t) = -\sum_{k} \log_2 p\big(\text{token}_k \mid \text{tokens before it}\big), \qquad H(B \mid A) = H(A+B) - H(A). \tag{3}$$

| | |
|---|---|
| $H(t)$ | the Shannon information of text $t$ under Qwen, in bits |
| $p(\text{token}_k \mid \cdot)$ | the probability Qwen gives token $k$ after reading the tokens before it |
| $H(B \mid A)$ | the bits B costs once A has been read |

Every token is scored, the first one included, by starting each text with Qwen's document separator.

None of the four relations holds for Shannon information (Tables 1 to 2). Every added token costs bits, so a longer text always carries more, and what costs most is what surprises Qwen: "salmon" costs 16.4 bits, "her" in "because he loves her" 0.4. Salient information is therefore not what Qwen's surprise measures.

## 2.2 Properties of the function that measures salient information

The data give the function $I$ three properties:

1. **It is not surprise.** Shannon information breaks all four relations (Table 2), so the function cannot be built from Qwen's token probabilities.

| text | $H$ (bits) |
|---|---|
| A | 63.4 |
| B | 50.8 |
| C | 72.6 |
| A + B | 92.1 |
| A + C | 111.9 |
| B, after A: $H(B \mid A)$ | 28.7 |
| C, after A: $H(C \mid A)$ | 48.6 |

Table 1: The Shannon information of every text under Qwen3-8B.

| salient information | Shannon information | holds |
|---|---|---|
| $\lvert V(A)\rvert \approx \lvert V(B)\rvert$ | $H(A) = 63.4$, $H(B) = 50.8$ | no: A costs a quarter more |
| $\lvert V(A)\rvert > \lvert V(C)\rvert$ | $H(A) = 63.4$, $H(C) = 72.6$ | no: C costs more |
| $\lvert V(A+B)\rvert \gg \lvert V(A+C)\rvert$ | $H(A+B) = 92.1$, $H(A+C) = 111.9$ | no: the reverse |
| $\lvert V(A)\rvert \approx \lvert V(A+C)\rvert$ | $H(A) = 63.4$, $H(A+C) = 111.9$ | no: A + C costs 76% more |

Table 2: The relations of salient information, tested with Shannon information.

2. **It is not additive over tokens.** $|V(A)| \approx |V(A+C)|$ although A + C has five more tokens than A, so the function must give C's points almost nothing while giving B's a lot.

3. **It is not distance.** B and C move the points by about the same amount: after layer 16, the last points of A + B and A + C lie equally far from A's last point (cosine 0.34 and 0.37, distance 96 and 95). They move them in different directions (Figure 2), so the function must tell directions apart.

Together: the function is a projection onto a subspace of Qwen's latent space that keeps the directions B's points move in and discards the directions C's points move in.

### 2.3 Movement

Figure 3 shows the points of A, B and C as areas with their centroids after layers 1, 10, 20 and 36.

Figure 4 runs A, B, C, A + B and A + C through Qwen separately and draws the area of each.

Adding B or C never moves A's points: inside A + B and A + C they stay within 2.4% of their length of where A alone puts them, which is rounding. Qwen reads left to right, so A's tokens cannot see what follows them. B's and C's tokens do see A, and A moves their points away from where they sit alone, by a share of their length that grows with depth and then falls:

| after layer | 1 | 10 | 16 | 20 | 36 |
|---|---|---|---|---|---|
| distance from where they sit alone | 16–29% | 51–86% | 64–84% | 59–81% | 24–59% |

Everything A + B means beyond A is therefore written into B's points. The areas in Figures 3 and 4 do not represent the meaning of a sentence: an area only shows how spread out one text's token points are. What reading A does to B is the drift: the gap between where B's points sit alone and where they sit inside A + B.

The areas do not measure salient information either:

1. **Their size does not grow with salient information.** From layer 10 on, A + C's points are more spread out than A + B's, although $|V(A+B)| \gg |V(A+C)|$.

2. **Their centroids do not tell B from C by distance.** A + B's and A + C's centroids lie equally far from A's centroid. They lie in different directions from it, as the last points do in the properties above, so the two centroids are as far from each other as from A's.

| after layer | 1 | 10 | 16 | 20 | 36 |
|---|---|---|---|---|---|
| spread of A + B | 10.2 | 43.4 | 54.5 | 71.0 | 59.6 |
| spread of A + C | 10.2 | 47.6 | 58.8 | 75.1 | 67.7 |
| distance of A + B's centroid from A's | 6.3 | 25.1 | 31.9 | 42.0 | 32.5 |
| distance of A + C's centroid from A's | 6.1 | 25.8 | 31.6 | 40.2 | 30.0 |
| distance between A + B's and A + C's centroids | 3.7 | 23.5 | 30.3 | 39.6 | 30.7 |

The spread of a text is the root mean square distance of its points from their centroid, measured in $\mathbb{R}^{4096}$, token 0 left out.

**Conclusion.** Nothing in Figures 3 and 4 or in these tables indicates that Qwen understands which information is salient.

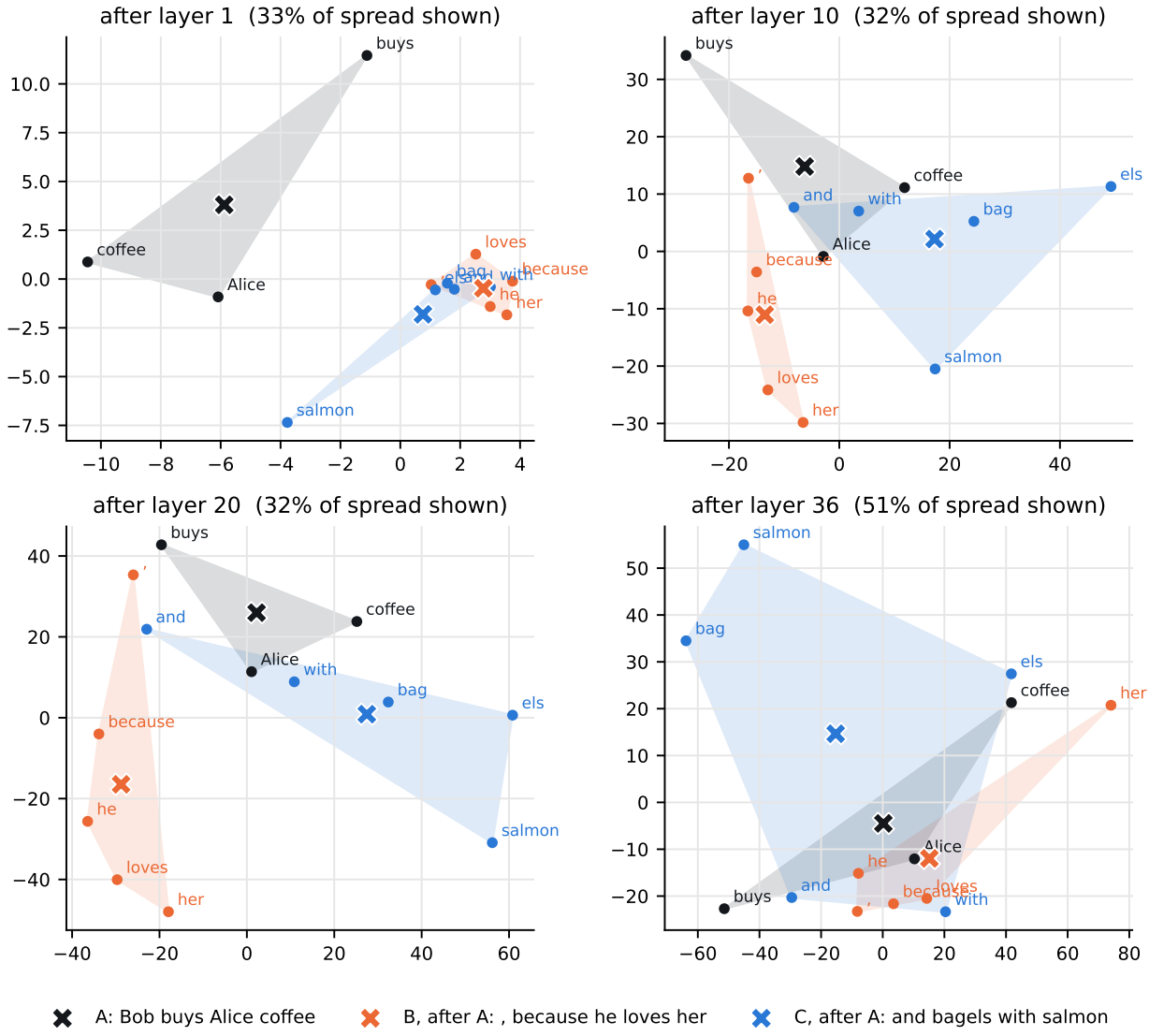



Figure 3: The points of A, A + B and A + C after layers 1, 10, 20 and 36, token 0 ("Bob") left out. Black: A's points. Orange: the points B adds after A. Blue: the points C adds after A. Each shaded area is the smallest polygon around one colour's points; the cross is their centroid. Each panel draws its layer's 13 points along the two directions in which they differ most, with the share of their spread those directions hold in the title.

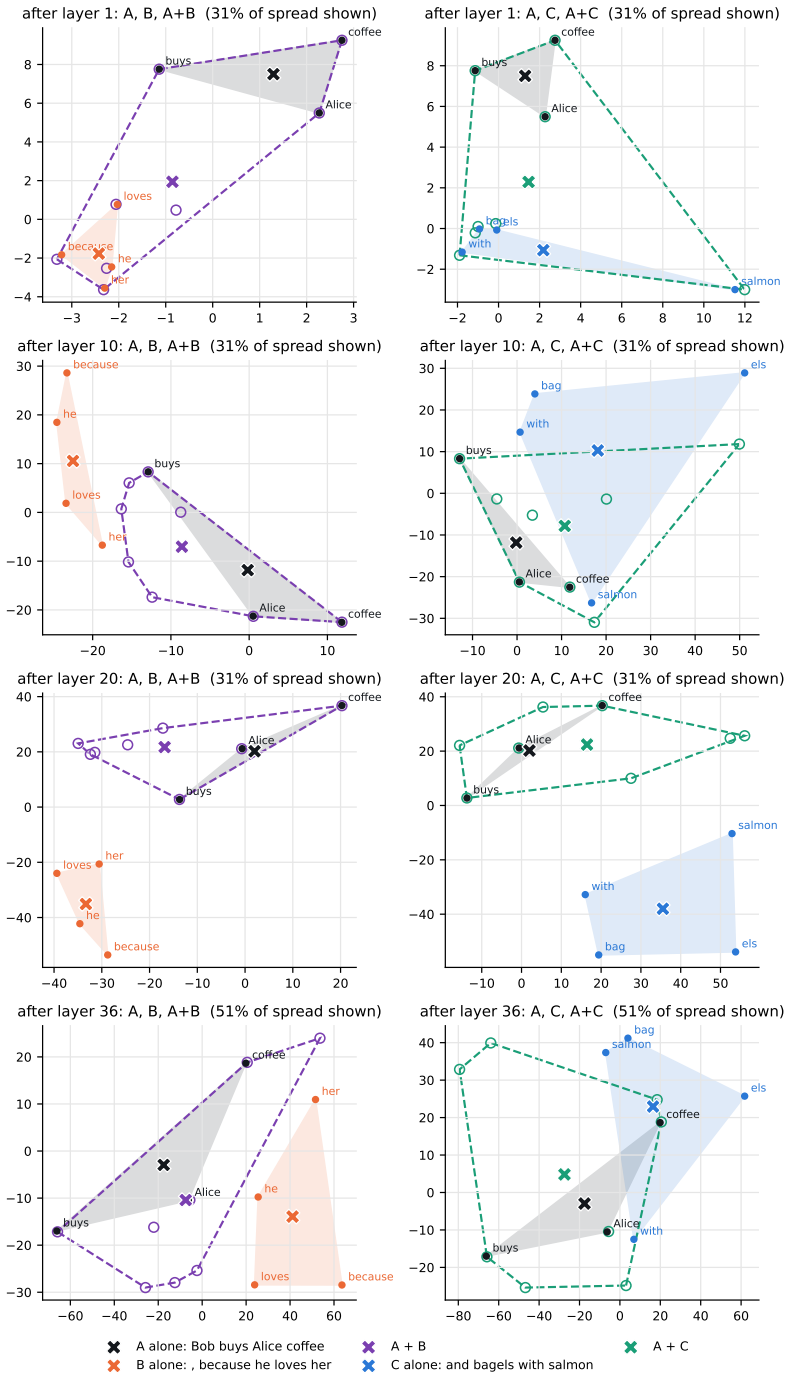



Figure 4: A, B, C, A + B and A + C, each run through Qwen on its own, after layers 1, 10, 20 and 36, every text's token 0 left out. Filled areas: A, B and C alone. Dashed outlines and hollow circles: A + B (left) and A + C (right). Crosses: centroids. Both panels of a row are drawn along the same two directions, those in which that layer's points differ most.

# 3 Transforming Qwen's output

## 3.1 Volumes that represent salient information

Figures 3 and 4 show the transformations Qwen applies to tokens: they project them into a generic latent space, shaped by the loss Qwen was trained with, next-token prediction. We want to fine-tune or adapt Qwen's output so that volumes begin to represent salient information. In the adapted space, $V(t)$ is the volume that surrounds text $t$.

$V(t) \cap V(t')$ the overlap of the volumes of texts $t$ and $t'$

**Requirement 6** (Size is salience)**.** *The larger the salient information, the larger the volume:* $I(t) > I(t') \implies |V(t)| > |V(t')|$.

**Requirement 7** (Overlap is shared salience)**.** *Volumes that overlap share salient information:* $V(t) \cap V(t')$ *is the salient information* $t$ *and* $t'$ *share.*

For example, with four texts

A "Bob buys Alice coffee"
B "because he loves her"
D "Charlie takes Jean to the movies"
E "where they share an intimate kiss"

the volumes must satisfy

$$\begin{aligned} V(A) \cap V(D) &= \emptyset, \\ V(A+B) \cap V(D+B) &= V(B), \\ V(A+B) \cap V(D+E) &= V(B) \cap V(E) \neq \emptyset, \\ V(A+E) \cap V(D+B) &= V(B) \cap V(E) \neq \emptyset. \end{aligned}$$

Figures 5 and 6 illustrate how these relations would look on transformed points.

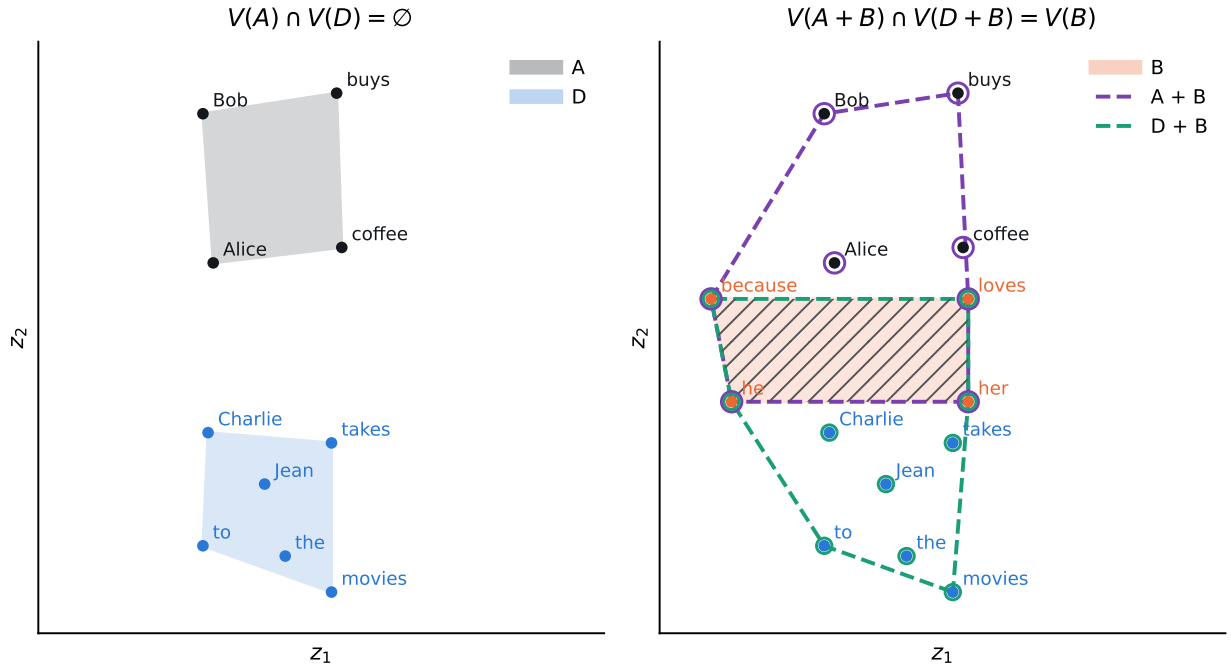



Figure 5: Illustration, not computed from data: the first two relations on transformed points. Each dot is the point of one token in the adapted space; $z_1$ and $z_2$ are two of its dimensions. A text's volume is the smallest polygon around its tokens' points. Filled areas: texts alone. Dashed outlines and hollow circles: joined texts, whose points are the points of their parts. Hatched: the overlap of the two dashed volumes, the left side of the relation. Left: $V(A) \cap V(D) = \emptyset$; A's and D's volumes do not meet. Right: $V(A + B) \cap V(D + B) = V(B)$; A + B and D + B overlap exactly in B's volume.

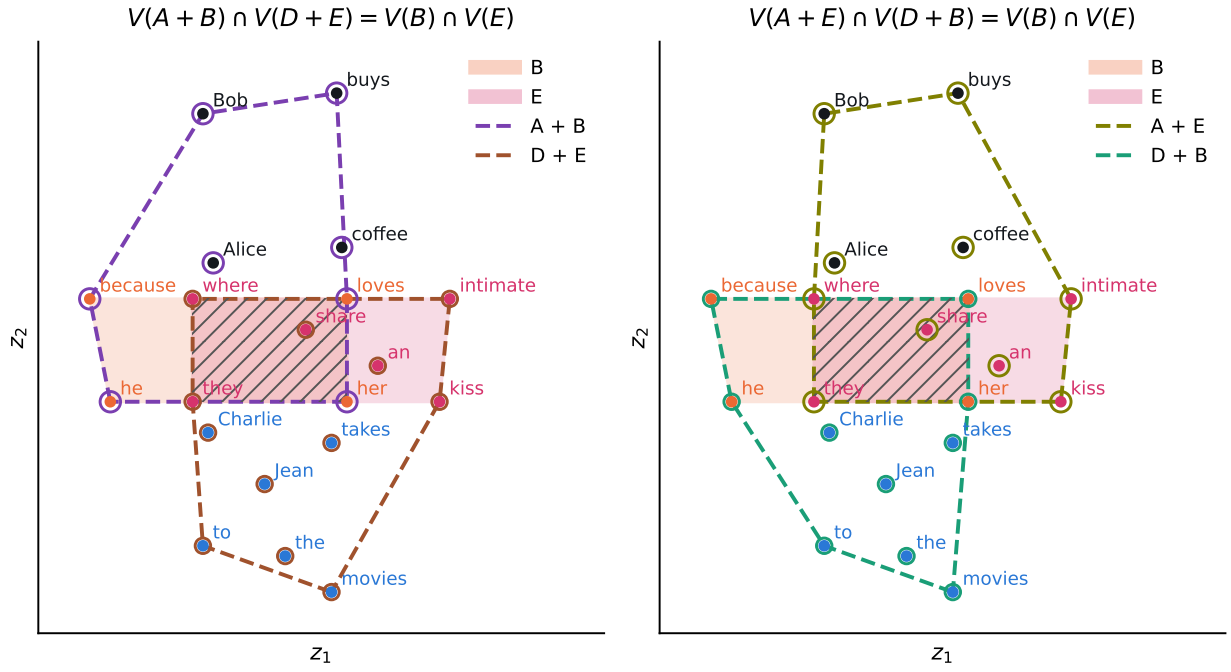



Figure 6: Illustration, not computed from data: the last two relations on transformed points, drawn as in Figure 5. B and E overlap. Left: $V(A+B) \cap V(D+E) = V(B) \cap V(E)$; A + B and D + E overlap exactly where B and E do. Right: $V(A+E) \cap V(D+B) = V(B) \cap V(E)$; so do A + E and D + B.

## 3.2 Alternatives we considered

### 3.2.1 A deterministic transformation

Could the transformation be computed without learning anything? No. The only deterministic, untrained signal we know of is how much a span moves Qwen's points for the text that follows it. To test it, we put a span after A and a follow-up sentence after the span, and measured how far the follow-up's points move compared with A followed directly by the same follow-up: 1 minus the mean cosine between the two sets of points, averaged over five follow-ups ("Alice smiled.", "She thanked him.", "Then they sat down.", "Alice looked at him.", "She took a sip.").

| | span after A | $1 - \cos$ after layer 16 |
|---|---|---|
| motives | ", because he loves her" | 0.14 |
| | ", since he misses her" | 0.14 |
| | ", hoping to win her" | 0.21 |
| details | "and bagels with salmon" | 0.09 |
| | "with oat milk and sugar" | 0.07 |
| | "and two glasses of water" | 0.07 |
| | ", and he pays cash" | 0.12 |
| | ", with his left hand" | 0.14 |

The signal falls short in two ways:

1. **It does not separate cleanly.** The motives move the follow-ups more than the food details do, but "and he pays cash" and "with his left hand" move them almost as much as the motives.

2. **It gives only a size.** It does not say which directions two texts share, and Requirement 7 needs exactly that.

Choosing the subspace that keeps B's directions and discards C's (Section 2.2) needs examples of what is salient. That is learning, even when it is done in closed form rather than by gradient descent.

### 3.2.2 How to train

Qwen does not understand which information is salient (Section 2.3), so every approach learns it from our labelled texts and maps it into a new space. They differ only in where the learned part sits. Qwen provides the text understanding; the learned part learns which information is salient. An approach can be used if its learned part is trained by minimising our loss, leaves Qwen's text understanding intact, and has few enough parameters for our few labelled examples. Among those that can, what the learned part sees decides how much it can learn:

| approach | trained by our loss | Qwen kept intact | trained parameters | learned part sees | usable |
|---|---|---|---|---|---|
| LoRA adapter | yes | yes | 44 million | every layer of Qwen | yes |
| soft prompt | yes | yes | 82 thousand | every layer, through its vectors | yes |
| fully connected head | yes | yes | 262 thousand | Qwen's finished points | yes |
| closed-form head | no | yes | none | Qwen's finished points | no |
| retraining all of Qwen | yes | no | 8.2 billion | every layer of Qwen | no |

The soft prompt is counted as 20 vectors of 4096 numbers. The closed-form head fails the first test; retraining all of Qwen fails the second and the third. The head sees only Qwen's finished points, so it can learn salience only if salience is a simple function of them, and nothing indicates that it is (the conclusion of Section 2.3). LoRA works inside every layer of Qwen and has no such condition, so it comes first. The soft prompt also changes what every layer computes, through its vectors, so it comes before the head too.

Sorted from the best option to start with:

| approach | mechanism | applicable | because |
|---|---|---|---|
| LoRA adapter | learned changes inside Qwen's layers, then a map into the new space | yes | it changes what Qwen's layers compute, so it does not depend on Qwen already computing what salience is judged from. |
| soft prompt | learned vectors before the text steer Qwen's layers | yes | like LoRA, it changes what every layer of Qwen computes, so it does not depend on Qwen already computing what salience is judged from; but it is weaker, because it changes only what the layers read, not their weights. |
| fully connected head | a learned map from Qwen's frozen points into the new space | yes | cheaper than LoRA and leaves Qwen untouched, but works only if salience is a simple function of Qwen's finished points, which nothing indicates (the conclusion of Section 2.3). |
| closed-form head | a map computed by a formula from labelled texts | no | a formula, such as a difference of means, cannot be fitted to the overlap relations; only minimising a loss can |
| retraining all of Qwen | all of Qwen's weights learned anew | no | our few labelled examples cannot constrain 8.2 billion parameters: it would overfit them and erase Qwen's knowledge of language, and costs about 130 GB to train and a second copy of Qwen to serve |

### 3.2.3 To do

| approach | to do |
|---|---|
| LoRA adapter | 1. Train the adapter and a head on the labelled texts with the salient point classification loss. 2. Serve with the adapter switched on per request. |
| soft prompt | 1. Train the vectors and a head on the labelled texts with the salient point classification loss. 2. Add an endpoint that accepts vectors in place of tokens. |
| fully connected head | 1. Run every labelled text through Qwen once and store its points. 2. Train the head on them with the salient point classification loss. 3. Test the relations on texts it was not trained on. |

## 3.3 Training data

**Schema.**

```
{"data": [
  {
    "file": "my_file.author",
    "concepts": {
      "love": 1,
      "hate": 2,
      "Michael": 3,
      ...
```

```
    },
    "annotations": [
      {"start_word": 13, "end_word": 25, "concept": 1},
      {"start_word": 18, "end_word": 22, "concept": 2},
      ...
    ]
  },
  ...
]}
```

`start_word` and `end_word` index the author document's words: its prose numbered from 0, in reading order across all its cells, a word being a run of characters between whitespace that contains a letter or a digit.

### 3.4 LoRA parameters and justification

| setting | value | justification |
|---|---|---|
| rank $r$ | 1 | data needed $D \propto N^{-\alpha/\beta}$ with $\lvert\alpha/\beta\rvert \leq 0.18$ [3]: it does not depend on the adapter's size, so the smallest rank |
| layers | all 7 linear maps of all 36 layers | attention-only LoRA underperforms [2] |
| trained parameters $N$ | 2,727,936 | $36 \cdot \sum_{m=1}^{7}(d_m^{\text{in}} + d_m^{\text{out}}) \cdot r = 36 \cdot 75{,}776 \cdot 1$ |
| $\alpha$ | 32 | [2] |
| learning rate | 10× full fine-tuning | fitted multiplier 9.8 [2] |
| batch size | small | LoRA tolerates large batches worse [2] |

### 3.5 Dataset size

**Dataset.** 10 stories, 10,000 words each; about 1,780 annotated spans per story.

**Metric: pair accuracy.** Take two annotated spans from a story the adapter was not trained on. The adapter is right when their volumes overlap exactly when the two spans share a salient concept. Pair accuracy is the share of such pairs it gets right, over equal numbers of pairs that share a concept and pairs that do not, so guessing scores 0.5.

**Predicted value.** It cannot be predicted from published results: the smallest dataset they report, 1,000 clinical conversations of about 3,700 words each [1], holds 37 times more text than these 10 stories. It has to be measured: train on 9 stories, measure pair accuracy on the 10th, and repeat for each of the 10.

### 3.6 The story space

| trained part | parameters |
|---|---:|
| LoRA adapter (Section 3.4) | 2,727,936 |
| layer weights $\alpha$ | 37 |
| query $q$ | 4,096 |
| projection $W$ | 262,144 |
| cap radius $\theta$ | 1 |
| cap sharpness $\kappa$ (Section 3.7) | 1 |

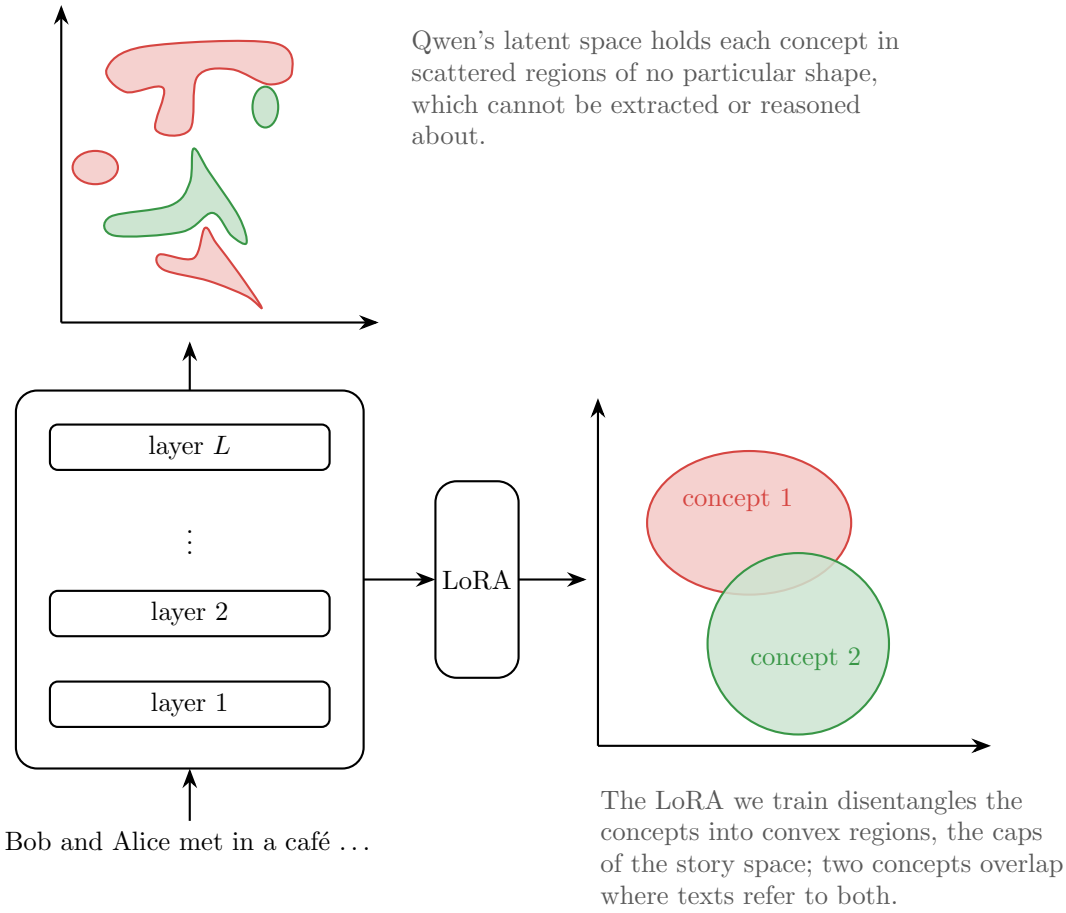



Figure 7: Illustration, not computed from data. A story's text runs through Qwen's layers. In Qwen's own latent space (top), the texts about a concept land in scattered regions of no particular shape, interleaved with other concepts' regions. The LoRA we train changes what the layers compute so that in the story space (right) each concept is one convex region, its cap (Section 3.6). The salient point classification loss is what moves each text into the caps of the concepts it refers to.

### 3.7 The salient point classification loss

Qwen does not model anything as manifolds. The manifolds are implied, but the latent space cannot be decompiled to see how they are shaped: all Qwen gives is a cloud of points in its latent space.

The goal is, given a concept's name, its abstract and sample points, to change how the model projects the relevant points so that they form a convex, contiguous area in the space.

The loss therefore:

1. embeds the abstract of a concept;
2. embeds all texts that are examples of that concept.

This gives a set of points in the latent space. The loss then measures:

- whether those points form a convex area;
- whether the point of the concept's abstract is the centroid of that area.

**The construction of the LoRA latent space.** Every text is run through Qwen with the LoRA adapter, which gives each of its tokens a state after every layer. Each token's states are mixed over the layers with learned weights $\alpha$ (4). The tokens are then pooled into one state, weighted by how much a learned query $q$ attends to each of them, leaving out token 0, the attention sink (5). A learned matrix $W$ projects that state into the $d$ dimensions of the LoRA latent space (6), and the projection is embedded on the unit sphere (7). Each concept is a hypersphere in that space. Its centre is the embedding of the concept's abstract (8), and its radius is the distance from that centre to the farthest of the concept's examples $E_c$ (9), so every example lies inside it. A single example is enough to set the radius. The concept's area $B_c$ is the part of the unit sphere that lies inside its hypersphere (10).

For a text $t$ of $n$ tokens, and for the concepts $C$ of a story, each with its abstract $A_c$ and its examples $E_c$:

$$h_k(t) = \sum_{\ell=0}^{L} \mathrm{softmax}(\alpha)_\ell \, h_{\ell,k}(t), \tag{4}$$

$$g(t) = \sum_{k=1}^{n-1} \mathrm{softmax}_k\big(q \cdot h_k(t)\big) \, h_k(t), \tag{5}$$

$$y(t) = W g(t), \tag{6}$$

$$z(t) = \frac{y(t)}{\|y(t)\|}, \tag{7}$$

$$m_c = z(A_c), \tag{8}$$

$$r_c = \max_{e \in E_c} \|z(e) - m_c\|, \tag{9}$$

$$B_c = \{\, z \in \mathbb{R}^d : \|z\| = 1,\ \|z - m_c\| \le r_c \,\}. \tag{10}$$

| | |
|---|---|
| $h_{\ell,k}(t)$ | the state of token $k$ of text $t$ after layer $\ell$ of Qwen with the LoRA adapter |
| $\alpha$ | the learned layer weights, one per layer |
| $h_k(t)$ | token $k$'s states, mixed over the layers |
| $q$ | the learned query that weighs the tokens, in $\mathbb{R}^D$ |
| $g(t)$ | the state of text $t$, pooled over its tokens |
| $W$ | the learned projection into the LoRA latent space, $d \times D$ |
| $C$ | the concepts of the story |
| $E_c$ | the examples of concept $c$ |
| $y(t)$ | the projection of text $t$ into the LoRA latent space |
| $z(t)$ | the embedding of text $t$, on the unit sphere |
| $A_c$ | the abstract of concept $c$ |
| $m_c$ | the centre of $c$'s hypersphere: the embedding of its abstract |
| $r_c$ | the radius of $c$'s hypersphere |
| $B_c$ | the area of $c$: the part of the unit sphere inside its hypersphere |

**The loss.** The loss (14) quantifies whether a concept's examples form a convex area in the LoRA latent space, uniformly distributed, with the concept's abstract at its centre. Every example lies inside its hypersphere by construction, so the loss measures how many of the story's unrelated texts $U_c$ lie outside it (11), (12). It measures how evenly the examples are spread over the area $B_c$, by their energy distance to points drawn uniformly from it (13); a concept with a single example has no spread to measure, and the term is 0. The distance is 0 only when the examples are spread over $B_c$ as evenly as those points are. In $d$ dimensions most of an area like $B_c$ lies near its edge, so evenly spread examples lie mostly near the edge too.

The unrelated texts $U_c$ are the negative examples of $c$: texts the author tagged as examples of the story's other concepts, but not of $c$. They are a random sample of those texts, drawn anew at every training step.

For each concept $c$, with the story's texts unrelated to it $U_c$:

$$P(t \in c) = \sigma\big(\kappa\,(r_c^2 - \|z(t) - m_c\|^2)\big), \qquad \kappa = 50, \tag{11}$$

$$\mathcal{L}_{\text{class}}(c) = \frac{1}{|U_c|} \sum_{u \in U_c} P(u \in c), \tag{12}$$

$$\mathcal{L}_{\text{spread}}(c) = 2\,\mathbb{E}\,\|z(e) - w\| - \mathbb{E}\,\|z(e) - z(e')\| - \mathbb{E}\,\|w - w'\|, \tag{13}$$
$$e, e' \in E_c,\ e \neq e', \quad w, w' \text{ drawn uniformly from } B_c,\ w \neq w',$$

$$\mathcal{L} = \frac{1}{|C|} \sum_{c \in C} \Big(\mathcal{L}_{\text{class}}(c) + \gamma\,\mathcal{L}_{\text{spread}}(c)\Big). \tag{14}$$

| | |
|---|---|
| $U_c$ | the story's texts unrelated to $c$ |
| $\kappa$ | how steeply $P(t \in c)$ drops across the hypersphere's surface |
| $\sigma$ | the logistic function, $\sigma(x) = 1/(1 + e^{-x})$ |
| $P(t \in c)$ | the probability that text $t$ lies inside $c$'s hypersphere |
| $w,\ w'$ | points drawn uniformly from $B_c$ |
| $\mathbb{E}$ | the mean over the examples in $E_c$ and the drawn points |
| $\gamma$ | the weight of the spread term |

# References


[1] Baseten. Practical LoRA research. October 2025. `labs.baseten.co/articles/practical-lora-research`.

[2] J. Schulman and Thinking Machines Lab. LoRA Without Regret. 2025. `thinkingmachines.ai/blog/lora`.

[3] B. Zhang, Z. Liu, C. Cherry and O. Firat. When Scaling Meets LLM Finetuning: The Effect of Data, Model and Finetuning Method. ICLR 2024. arXiv:2402.17193.